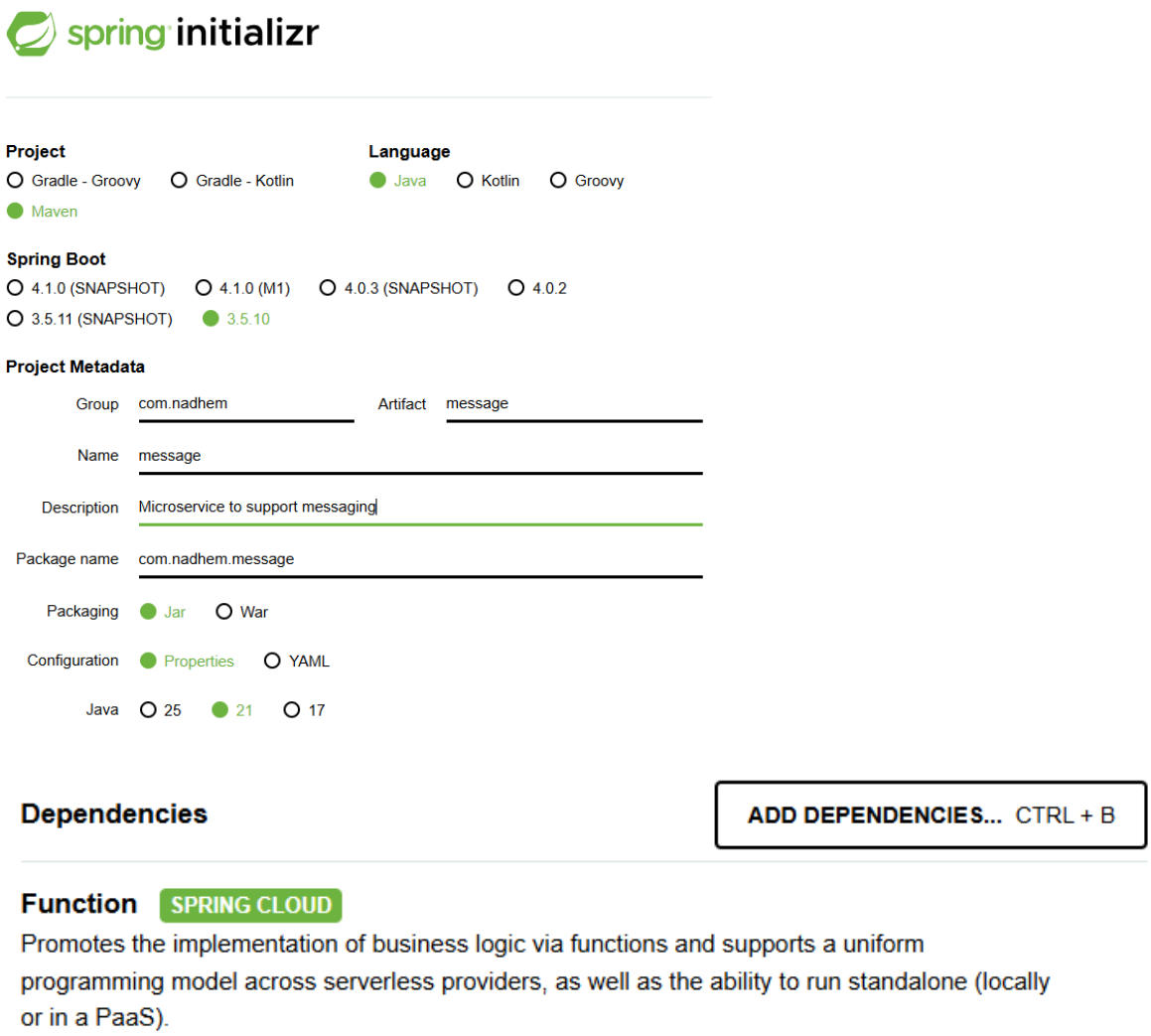
Event driven microservices utilisant

RabbitMQ et Spring Cloud Functions & Stream

Objectifs :

1. Créer le microservice messsage
2. Créer le dto et la classe MessageFunctions
3. Convertir les fonctions en api REST
4. Modifier les microservices message & teacher pour l'intégration avec RabbitMQ
5. Créer l'api createTeacher
6. Tester la communication asynchrone en utilisant RabbitMQ et event streaming
7. Etablir une communication asynchrone à partir de message vers teacher
8. La communication asynchrone en utilisant RabbitMQ et event streaming et docker compose

Créer le microservice messsage



Créer le dto et la classe MessageFunctions

1. Créer dans le package dto, le record *TeacherMessageDto*

```
package com.nadhem.message.dto;

public record TeacherMessageDto(Long teacherId, String email,

String mobileNumber) {
}
```

2. Créer la classe MessageFunctions

```
package com.nadhem.message.functions;

import com.nadhem.message.dto.TeacherMessageDto;
```

```java
import org.slf4j.Logger;
import org.slf4j.LoggerFactory;
import org.springframework.context.annotation.Bean;
import org.springframework.context.annotation.Configuration;

import java.util.function.Function;

@Configuration
public class MessageFunctions {

   private static final Logger log =
LoggerFactory.getLogger(MessageFunctions.class);

    @Bean
    public Function<TeacherMessageDto,TeacherMessageDto> email() {
        return teacherMsgDto -> {
            log.info("Sending email with the details : " +
teacherMsgDto.toString());
            return teacherMsgDto;
        };
    }

    @Bean
    public Function<TeacherMessageDto,Long> sms() {
        return teacherMsgDto -> {
            log.info("Sending sms with the details : " +
teacherMsgDto.toString());
            return teacherMsgDto.teacherId();
        };
    }

}
```

Convertir les fonctions en api REST

3. Ajouter la dépendance suivante :

```xml
<dependency>
    <groupId>org.springframework.cloud</groupId>
    <artifactId>spring-cloud-starter-function-web</artifactId>
</dependency>
```

4. Tester avec Postman

```
{"teacherId" : 1,
"email" : "nadhemb@yahoo.com",
"mobileNumber" : "123456"
}
```

http://localhost:8080/email [POST]

application.yml

```yaml
server:
  port: 9010

spring:
  application:
    name: "message"
  cloud:
    function:
      definition: email|sms
```

http://localhost:9010/emailsms

Pourquoi utiliser Spring Cloud Stream

https://spring.io/projects/spring-cloud-stream

Modifier les microservices message & teacher pour l'intégration avec RabbitMQ

5. Au niveau du microservice message, supprimer toutes les dépendances et remplacer les par :

```xml
<dependency>
  <groupId>org.springframework.boot</groupId>
  <artifactId>spring-boot-starter</artifactId>
</dependency>
<dependency>
  <groupId>org.springframework.cloud</groupId>
  <artifactId>spring-cloud-stream</artifactId>
</dependency>
<dependency>
  <groupId>org.springframework.cloud</groupId>
  <artifactId>spring-cloud-stream-binder-rabbit</artifactId>
</dependency>
<dependency>
  <groupId>org.springframework.boot</groupId>
  <artifactId>spring-boot-starter-test</artifactId>
  <scope>test</scope>
</dependency>
```

6. Ajouter la prop stream au fichier application.yml

```
cloud:
  function:
    definition: email|sms
  stream:
    bindings:
      emailsms-in-0:
        destination: send-communication
        group: ${spring.application.name}
```

7. Ajouter la prop rabbitMQ au fichier application.yml (sous spring)

```
rabbitmq:
  host: localhost
  port: 5672
  username: guest
  password: guest
  connection-timeout: 10s
```

Modifier le microservice Teacher

Au niveau du microservice teacher, ajouter les dépendances :

```
<dependency>
    <groupId>org.springframework.cloud</groupId>
    <artifactId>spring-cloud-stream</artifactId>
</dependency>
<dependency>
    <groupId>org.springframework.cloud</groupId>
    <artifactId>spring-cloud-stream-binder-rabbit</artifactId>
</dependency>
```

8. Au niveau du microservice teacher, créer la dto TeacherMessageDto :

```
package com.nadhem.teacher.dto;

public record TeacherMessageDto(Long teacherId, String email, String mo-
bileNumber) {
}
```

9. Au niveau du microservice teacher, ajouter la prop stream au fichier application.yml (même niveau que openfeign)

```
cloud:
  stream:
```

```
    bindings:
        sendCommunication-out-0:
         destination: send-communication
  openfeign:
…..
```

10. Au niveau du microservice teacher, Ajouter la prop rabbitMQ au fichier application.yml (sous spring)

```
rabbitmq:
  host: localhost
  port: 5672
  username: guest
  password: guest
  connection-timeout: 10s
```

Créer l'api createTeacher

Ajouter ces attributs à l'entité Teacher et à la dto TeaherDto:

```
private String email;
private String mobileNumber;
```

Ajouter la méthode createTeacher à l'interface TeacherService

```
void createTeacher(TeacherDto teacherDto);
```

Implémenter la méthode dans la classe TeacherServiceImpl :

```
private static final Logger log = LoggerFactory.getLogger(TeacherServiceImpl.class);
private final StreamBridge streamBridge;
```

```
@Override
public void createTeacher(TeacherDto teacherDto) {

  Teacher savedTeacher = teacherRepository.save(Teacher.builder()
      .firstName(teacherDto.getFirstName())
      .lastName(teacherDto.getLastName())
      .depCode(teacherDto.getDepCode())
      .email(teacherDto.getEmail())
      .mobileNumber(teacherDto.getMobileNumber())
      .build());

  sendCommunication(savedTeacher);
}


private void sendCommunication(Teacher teacher) {
    var teacherMsgDto = new TeacherMessageDto(teacher.getId(),
```

```
teacher.getEmail(),teacher.getMobileNumber());
    log.info("Sending Communication request for the details: {}", teach-
erMsgDto);
    var result = streamBridge.send("sendCommunication-out-0", teacherMs-
gDto);
    log.info("Is the Communication request successfully triggered ? : {}",
result);
}
```

Ajouter l'api create Teacher :

```
@PostMapping("/create")
public ResponseEntity<String> createTeacher(@RequestBody TeacherDto
teacherDto) {
    teachersService.createTeacher(teacherDto);
    return ResponseEntity
            .status(HttpStatus.CREATED)
            .body("Teacher created successfully");
}
```

Tester la communication asynchrone en utilisant RabbitMQ et event streaming

11. Démarrer RabbitMQ en tant que docker container

docker run -d -it --rm --name rabbitmq -p 5672:5672 -p 15672:15672 rabbitmq:4.0-management

12. Démarrer tous les services et tester la création d'un teacher avec Postman

Commande de démarrage de Keycloack e tant que docker container :

docker run -d -p 8090:8080 -e KC_BOOTSTRAP_ADMIN_USERNAME=admin -e KC_BOOTSTRAP_ADMIN_PASSWORD=admin quay.io/keycloak/keycloak:26.3.4 start-dev

test postman

```
{"firstName":"Nadhem",
"lastName":"Bel Hadj",
"depCode":"IT",
"email": "nadhemb@yahoo.com",
"mobileNumber":"123456"
}
```

Etablir une communication asynchrone à partir de message vers teacher

Message/application.yml

```yaml
stream:
  bindings:
    emailsms-in-0:
      destination: send-communication
      group: ${spring.application.name}
    emailsms-out-0:
      destination: communication-sent
```

Création de la classe *TeacherFunctions* :

SeviceTeacher

```
boolean updateCommunicationStatus(Long teacherId);
```

ServiceTeacherImpl

```
@Override
public boolean updateCommunicationStatus(Long teacherId) {
   boolean isUpdated = false;
   if(teacherId !=null ){
      Teacher teacher = teacherRepository.findById(teacherId).orElseThrow(
            () -> new ResourceNotFoundException("Teacher", "TeacherId", teacherId.toString())

      );
      teacher.setCommunicationDone(true);
      teacherRepository.save(teacher);
      isUpdated = true;
   }
   return  isUpdated;
}
```

```
package com.nadhem.teacher.exceptions;

import org.springframework.http.HttpStatus;
import org.springframework.web.bind.annotation.ResponseStatus;

@ResponseStatus(value = HttpStatus.NOT_FOUND)
public class ResourceNotFoundException extends RuntimeException {

   public ResourceNotFoundException(String resourceName, String fieldName, String fieldValue) {
      super(String.format("%s not found with the given input data %s : '%s'", resourceName, fieldName,
fieldValue));
   }
}
```

```java
package com.nadhem.teacher.functions;

import com.nadhem.teacher.service.TeacherService;
import org.slf4j.Logger;
import org.slf4j.LoggerFactory;
import org.springframework.context.annotation.Configuration;
import org.springframework.context.annotation.Bean;
import java.util.function.Consumer;

@Configuration
public class TeacherFunctions {
   private static final Logger log = LoggerFactory.getLogger(TeacherFunctions.class);

   @Bean
   public Consumer<Long> updateCommunication(TeacherService teacherServiceService) {
      return teacherId -> {
         log.info("Updating Communication status for the teacher Id : " + teacherId.toString());
         teacherServiceService.updateCommunicationStatus(teacherId);

      };
   }
}
```

teacher/application.yml

```yaml
cloud:
 function:
   definition: updateCommunication
```

```yaml
bindings:
   sendCommunication-out-0:
    destination: send-communication
   updateCommunication-in-0:
     destination: communication-sent
     group: ${spring.application.name}
```

La communication asynchrone en utilisant RabbitMQ et event streaming et docker compose

13. Générer le .jar des microservices message et teacher
14. Créer Dockerfile pour message

```
#Start with a base image containing Java runtime
FROM eclipse-temurin:21-jdk

#Information around who maintains the image
LABEL "org.opencontainers.image.authors"="nadhem.com"

# Add the application's jar to the image
COPY target/message-0.0.1-SNAPSHOT.jar message-0.0.1-SNAPSHOT.jar


# execute the application
ENTRYPOINT ["java", "-jar", "message-0.0.1-SNAPSHOT.jar"]
```

Docker-compose.yml

```
rabbit:
  image: rabbitmq:4-management
 container_name: rabbit
 ports:
   - "5672:5672"
   - "15672:15672"
  healthcheck:
    test: rabbitmq-diagnostics check_port_connectivity
```

```
teacher-ms:
  ....
environment:
...
     - SPRING_RABBITMQ_HOST=rabbit
  depends_on:
    rabbit:
      condition: service_healthy
...
```

```
message:
  build: ./message
  container_name: message-ms
  environment:
    - SPRING_RABBITMQ_HOST=rabbit
  depends_on:
    rabbit:
     condition: service_healthy
```

15. Arrêter tous des services au niveau de l'ide
16. Supprimer tous les containers
17. docker compose up -d
18. Recréer le client dept-app au niveau de Keycloak
19. Tester

Fichier docker-compose.yml

```
services:
  rabbit:
    image: rabbitmq:4.0-management
    #hostname: rabbitmq
    container_name: rabbit
    ports:
      - "5672:5672"
      - "15672:15672"
    healthcheck:
      test: rabbitmq-diagnostics check_port_connectivity
  eurekaserver:
    build: ./eurekaserver
    container_name: eurekaserver
#    ports:
#      - "8761:8761"
    healthcheck:
      test: "curl --fail --silent localhost:8761/actuator/health | grep UP || exit 1"
      interval: 10s

  configserver:
    build: ./configserver
    container_name: configserver
#    ports:
#      - "9999:9999"
    healthcheck:
      test: "curl --fail --silent localhost:9999/actuator/health | grep UP || exit 1"
      interval: 10s

  departments-ms:
    build: ./department-microservice
    container_name: departments-ms
    environment:
      - DISCOVERY_SERVICE_URL=http://eurekaserver:8761/eureka
      - CONFIG_SERVICE_URL=http://configserver:9999
```

```
    healthcheck:
      test: "curl --fail --silent localhost:8080/actuator/health | grep UP || exit 1"
    depends_on:
      eurekaserver:
        condition: service_healthy
      configserver:
        condition: service_healthy

  postgres:
    image: postgres:16
    container_name: kc-postgres-dev
    environment:
      POSTGRES_DB: keycloak
      POSTGRES_USER: keycloak
      POSTGRES_PASSWORD: keycloak
    volumes:
      - kc_pg_data:/var/lib/postgresql/data
    restart: unless-stopped

  keycloak:
    image: quay.io/keycloak/keycloak:26.3.4
    container_name: keycloak
    command: start-dev
    environment:
      - KC_DB=postgres
      - KC_DB_URL=jdbc:postgresql://postgres:5432/keycloak
      - KC_DB_USERNAME=keycloak
      - KC_DB_PASSWORD=keycloak
      - KC_BOOTSTRAP_ADMIN_USERNAME=admin
      - KC_BOOTSTRAP_ADMIN_PASSWORD=admin
    ports:
      - "8090:8080"
    depends_on:
      - postgres

  gatewayserver:
    build: ./gatewayserver
    container_name: gatewayserver
    ports:
      - "8888:8888"
    environment:
      - DISCOVERY_SERVICE_URL=http://eurekaserver:8761/eureka
      - SPRING_SECURITY_OAUTH2_RESOURCESERVER_JWT_JWK_SET_URI=http://keycloak:8080/realms/mas-
ter/protocol/openid-connect/certs
    healthcheck:
      test: "curl --fail --silent localhost:8888/actuator/health | grep UP || exit 1"
    depends_on:
      eurekaserver:
        condition: service_healthy
      keycloak:
        condition: service_started


  teacher-ms:
    build: ./teacher
    container_name: teacher-ms
    environment:
```

```
    - DISCOVERY_SERVICE_URL=http://eurekaserver:8761/eureka
    - CONFIG_SERVICE_URL=http://configserver:9999
    - SPRING_RABBITMQ_HOST=rabbit
   healthcheck:
    test: "curl --fail --silent localhost:8081/actuator/health | grep UP || exit 1"
   depends_on:
    rabbit:
     condition: service_healthy
    eurekaserver:
     condition: service_healthy
    configserver:
     condition: service_healthy
    departments-ms:
     condition: service_healthy
  message:
   build: ./message
   container_name: message-ms
   depends_on:
    rabbit:
     condition: service_healthy
   environment:
    - SPRING_RABBITMQ_HOST=rabbit



volumes:
  kc_pg_data:
```